软件工程基础

详细考试目录

（北京航空航天大学软件学院　杨溢龙）

全册共 186 页 | 8 章内容 | 按 PDF 页码标注

第 1 章：软件与软件工程 （第 1–17 页）

Part 1 软件 （第 1–7 页）

1.1 软件的基本概念与特点 （第 1 页）

- 软件的基本概念
- 软件的概念及特点
- 软件的分类
- 软件的地位和作用
- 建设狗窝和建设大厦的区别
- 算法 vs 程序 vs 软件
- 何为程序？——程序的两种形式（源代码/可执行代码）

1.2 程序编写与软件开发 （第 2 页）

- 程序如何编写？
- 软件开发难点与挑战
- 从程序到软件——工程开发的启发
- 目标领域类型（金融、信息技术、医疗等）
- 利益相关者（领域专家、设计师、架构师、开发/测试/运维工程师）
- 编程语言种类（C/C++/Java/Python/Dart/Go/Rust）
- 运行平台（Web/Desktop/Mobile）
- 系统类型（信息系统、嵌入式、CPS、区块链、AI 使能系统）

1.3 软件的构成：程序 + 数据 + 文档 （第 3 页）

- 软件开发的再认识
- 软件开发核心问题（什么是对？什么是好？）
- 从需求到代码的分阶段跨越
- 何为软件？（程序 + 数据 + 文档）
- 何为数据？（初始化数据 + 测试数据 + 日常数据）
- 数据示例（用户、订单、交易、日志数据）
- 数据处理（表示、获取、存储、检索、分析）

1.4 软件文档 （第 3–4 页）

- 何为文档?（开发文档 + 管理文档 + 产品文档）
- 为什么需要文档?（记不住、理不清、讲不明）
- 编写文档目的（阐述清楚、发现问题、开展交流、促进管理）
- Agent 工具生成能力对比（ATM/TakeOutMS/LoanPS 案例）
- 集成测试成功率 vs 单元测试成功率 vs 错误迭代次数
- openspec / speckit / claude 对比数据
- 文档的重要性（企业成熟度的标志）
- 软件文档分类：开发文档、管理文档、产品文档
- 软件文档的覆盖关系（6 层：目标程序 → 源程序 → 详细设计 → 概要设计 → 需求规格 → 用户需求 → 合同）

1.5 软件文档规范与模版 （第 5 页）

- 软件文档规范（国军标 GB438B）
- 参考模版

1.6 软件的特点小结 （第 5 页）

- 逻辑性、设计开发、易变性、复杂性、缺陷的隐蔽性

1.7 软件质量 （第 5–6 页）

- 软件质量要素（1/3）：正确性、可靠性、健壮性、有效性
- 软件质量要素（2/3）：安全性、可维护性、可移植性、可重用性
- 软件质量要素（3/3）：可理解性、可信性、持续性、易用性、互操作性

1.8 软件分类 （第 6–7 页）

- 按功能分类：系统软件、支撑软件、应用软件
- 关键基础软件
- 开源软件（Open Source Software）
- 开源软件许可证：MIT、BSD、Apache、MPL、LGPL、GPL

Part 2 软件演化及危机 （第 7–10 页）

1.9 软件发展与挑战 （第 8 页）

- 软件发展与挑战
- 传统的个体作坊式软件开发
- 个体作坊式创作带来的问题
- 软件开发面临的挑战：开发成本高、缺陷密度高、需求增长快、成功概率低、生产率低

1.10 软件危机 （第 9–10 页）

- 软件危机的出现
- 软件危机（Software Crisis）定义与表现
- 示例：IBM 360 OS 软件开发
- 软件维护困难

- 质量难以保证
- 软件错误后果
- 软件灾难故事：ARIANE 5 火箭爆炸
- 软件灾难故事：特斯拉 Autopilot
- 软件灾难故事：波音 737 MAX
- 大模型灾难故事
- 软件危机产生根源总结
- 解决危机的途径——软件工程

Part 3 软件工程 （第 11–17 页）

1.11 软件工程基本概念 （第 11–12 页）

- 软件工程产生的动机
- 软件开发方式的改变
- 软件工程的定义（IEEE 1993）
- 软件工程的作用

1.12 软件工程的发展历程 （第 12–14 页）

- 20 世纪 50-60 年代：萌芽阶段
- 20 世纪 70 年代：结构化方法兴起
- 20 世纪 80 年代：面向对象方法出现
- 20 世纪 90 年代：UML 统一建模语言、敏捷方法
- 21 世纪前十年：敏捷普及、开源兴起
- 近十年（2011-至今）：DevOps、AI 赋能
- 生成式软件开发 / 大模型
- 氛围编程（Vibe Coding）
- 提示词工程（Prompt Engineering）
- 智能体（Agent）

1.13 软件工程的知识体系 （第 14 页）

- IEEE SWEBOK V3.0 / V4.0（软件工程知识体系指南）
- 软件工程的多学科交叉

1.14 软件工程的目标与三要素 （第 15 页）

- 软件工程的目标
- 软件工程的三要素：过程（Process）、方法学（Methodology）、工具（Tool）

1.15 软件工程基本原则 （第 16–17 页）

1. 抽象和建模（Abstraction and Modeling）
2. 模块化（Modularity）
3. 软件重用（Software Reuse）
4. 信息隐藏（Information Hiding）

5. 关注点分离（Separation of Concerns）
6. 分而治之（Divide and Conquer）
7. 双向追踪原则（Bidirectional Traceability）
8. 工具辅助（Tool Support）

1.16 软件工程的基本观点与小结 （第 17 页）

第 2 章：软件过程与项目管理 （第 18–29 页）

Part 1 软件过程概述 （第 18–20 页）

2.1 软件过程与生命周期 （第 18–19 页）

- 软件过程（Software Process）定义
- 软件过程模型（Software Process Model）
- 软件生命周期模型概论
- 需求分析（Requirement Analysis）
- 概要设计（Architecture Design）
- 详细设计（Detailed Design）
- 编程实现（Implementation）
- 集成测试（Integration Test）
- 确认测试（Validation Test）
- 边写边改模型（Code and Fix）——最原始的模型

Part 2 软件过程模型 （第 20–23 页）

2.2 瀑布模型 （第 21 页）

- 瀑布模型（Waterfall Model, 1970）
- 特点：阶段顺序执行、文档驱动
- 过程分解与精化的核心思想：增量 / 迭代
- 过程分解——功能维度
- 过程分解——时间维度

2.3 增量模型与快速原型 （第 22 页）

- 增量模型（Incremental Model）
- 快速原型模型（Rapid Prototyping）
- 螺旋模型（Spiral Model）——Boehm 1988

2.4 敏捷方法与 Scrum （第 22–23 页）

- 敏捷宣言（Agile Manifesto, 2001）——4 条价值观
- Scrum 过程
- Scrum 角色：Product Owner（产品负责人）、Scrum Master、Team（开发团队）
- Sprint（冲刺）概念

Part 3 过程模型选择策略 （第 23–24 页）

- 软件过程模型的选择原则
- 场景一：V 模型 / 外包项目
- 场景二：高风险项目——螺旋模型
- 场景三：复刻产品——增量模型
- 场景四：持续更新——迭代模型 / 敏捷
- 模型选择小结

Part 4 软件项目管理 （第 25–29 页）

2.5 软件项目概述 （第 25–26 页）

- 软件项目及其特点
- 软件项目的任务
- 软件项目实施方法——工程化
- 何为软件项目管理?（三要素：过程、人员、产品）
- 软件项目的管理要素

2.6 过程管理 （第 27–28 页）

- 过程定义
- 软件度量
- 成本预算
- 活动排序
- 软件项目计划
- 甘特图（Gantt Chart）——描述项目进度计划
- 项目跟踪
- 风险管理

2.7 产品管理 （第 28–29 页）

- 软件质量保证（SQA）
- 软件配置管理（SCM）
- 基线的概念（Baseline）
- 软件需求管理

2.8 人员管理 （第 29 页）

- 人员管理
- 软件项目团队

第 3 章 / 第 5 章：需求工程 （第 30–79 页）

Part 1 需求基本概念 （第 30–36 页）

3.1 需求的定义与重要性 （第 30–32 页）

- 软件开发的本质
- 需求重要性（价值所在 / 开发基础 / 验收标准）
- 空巢老人看护系统示例
- 软件需求定义（IEEE 1997 标准）
- 何为需求？——解读 IEEE 标准

3.2 利益相关者分析 （第 32–33 页）

- 软件系统的利益相关者（用户、客户、系统分析员、开发者）
- 软件需求及其利益相关方
- 利益相关者对软件需求差异和冲突

3.3 软件需求的特点 （第 34 页）

- 隐式性（隐含的、未明确表达的）
- 隐晦性（难以明确表达的）
- 多源性（来源多样）
- 易变性（随时间变化）
- 领域知识相关性
- 价值不均性（不同需求价值不同）

3.4 软件需求的质量要求 （第 34–35 页）

- 有价值、正确、完整、无二义性
- 可行、一致、可追踪、可验证

3.5 可行性分析 （第 35–36 页）

- 技术可行性
- 设备可行性
- 进度可行性
- 成本可行性
- 商业可行性
- 社会可行性（法律法规）
- 可行性研究报告

Part 2 需求工程 （第 36–41 页）

3.6 需求工程的定义与特点 （第 36–37 页）

- 需求工程的重要性
- 何为需求工程？（Requirements Engineering）
- 需求工程的特点

- ISO/IEC/IEEE 15288 / 12207 / 29148 标准
- 需求工程指南及参考书目（IREB、INCOSE、IEEE 830）

3.7 需求工程过程模型 （第 38–40 页）

- 螺旋模型——三层次三阶段
- 需求的三个层次：业务需求、用户需求、系统需求
- 系统需求：功能性需求 vs 非功能性需求
- 非功能性需求——ISO 25010 标准分类
- 需求工程三阶段：需求获取、需求表示/精化与规约、需求确认与验证

3.8 需求管理 （第 40–41 页）

- 软件需求的缺陷
- 软件需求变更管理
- 需求工程相关的 CASE 工具

Part 3 需求获取 （第 41–50 页）

3.9 需求获取的来源与方法 （第 41–43 页）

- 软件需求从何而来?
- 需求获取为什么难?
- 成立需求分析的联合工作小组（JRT）
- 需求获取的方法：
  - 访谈和会议
  - 调查问卷
  - 现场观摩
  - 分析业务资料
  - 群体化方法（Brainstorming / JAD）
  - 大语言模型推荐

3.10 需求获取的一般过程（1/4）——明确问题与边界 （第 44 页）

- 明确软件要解决的问题
- 课程实践创意示例：无人值守图书馆
- 课程实践创意示例：多无人机联合搜寻

3.11 需求获取的一般过程（2/4）——识别利益相关方与功能需求 （第 45–46 页）

- 识别利益相关方
- 获取软件的功能性需求
- 合理的软件需求
- 获取和构思非功能性需求

3.12 需求获取的一般过程（3/4）——描述初步软件需求 （第 46–50 页）

- 自然语言描述及不足
- 结构化自然语言描述

- 用户故事（User Story）——三要素：角色、功能、价值
- EARS（Easy Approach to Requirements Syntax）——简洁需求句法
- EARS 五类模板：Ubiquitous / Event-driven / State-driven / Unwanted / Optional
- 交互流程描述（场景描述）
- 基本流程（Basic Flow）
- 扩展流程（Extension Flow）——编号编写、异常处理、选择
- 撰写初步软件需求描述文档

3.13 需求获取的一般过程（4/4）——评审初步需求 （第 50 页）

- 评审初步软件需求
- 需求获取（总图）

Part 4 需求精化与规约 （第 51–76 页）

3.14 需求精化与优先级 （第 51–52 页）

- 为什么要精化初始软件需求
- 软件需求的精化与规约的任务
- 分析和确定软件需求优先级
- 分析软件需求重要性
- 示例：空巢老人看护软件的需求重要性分析
- 确定需求迭代先后次序

3.15 需求精化的核心问题 （第 53 页）

- 两层次跨越：从问题域到解域

3.16 面向对象需求建模 （第 54–55 页）

- 面向对象系统需求的建模方法
- 用例模型（行为视角）
- 领域概念模型（结构视角）
- 系统操作合约
- 一阶逻辑 / 对象约束语言（OCL）

3.17 UML 统一建模语言 （第 56–57 页）

- UML: Unified Modeling Language
- Unified（统一）、Modeling（建模）、Language（语言）
- UML 本质——是一种语言，不是方法学
- UML 与软件开发方法、过程的关系
- 面向对象建模语言
- 面向对象需求分析方法的优势和特色
- UML 应用范围
- UML 图（9 种图 / 5 类）
- UML 的视点及图：

  - 用例图（Use Case Diagram）
  - 静态图：类图、对象图、包图
  - 结构：构件图、部署图
  - 行为：状态图、活动图、通信图、顺序图

3.18 用例图（Use Case Diagram） （第 57–61 页）

- 参与者（Actor）——识别参与者
- 参与者之间的泛化关系
- 系统边界框
- 用例（Use Case）
- 用例的特征 / 特点
- 用例往往是动宾短语
- 用例独立和完备性
- 用例由参与者启动
- 用例要有可观测的执行结果
- 用例描述
- 用例的识别
- 执行者与用例间的关系
- 用例间的关系：
  - 包含（Include）——必须执行
  - 扩展（Extend）——可选执行
- 超市管理系统——用例模型示例

3.19 顺序图（Sequence Diagram） （第 62–64 页）

- 顺序图的概念
- 三种主要作用
- 顺序图的组成元素：对象、生命线、激活、消息
- 对象（Object）
- 生命线（Lifeline）
- 激活（Execution / Activation）
- 消息（Message）
- 同步消息（Synchronous）vs 异步消息（Asynchronous）
- 系统顺序图（需求）VS 对象顺序图（设计）——重要区分

3.20 面向对象的核心概念 （第 64–66 页）

- 对象（Object）
- 类（Class）——对象与类的关系
- 继承（Inheritance）
- 多态（Polymorphism）
- 覆盖/重写（Override）
- 重载（Overload）

- 消息（Message）
- 聚合（Aggregation）和组合（Composition）——整体-部分关系

3.21 领域概念模型 （第 66–67 页）

- 识别类与对象——系统实体的识别方法
- 识别属性
- 识别关联关系
- 关联名称 / 角色 / 导向性
- 关联类
- 关联的多重性（1:1, 1:n, n:m）
- 领域模型（Domain Model）
- 小结

3.22 系统操作合约 （第 68–69 页）

- 系统操作（System Operation）
- 系统操作合约（System Operation Contract）
- 操作签名、交叉引用、前置条件、后置条件
- 面向对象的基本操作（创建对象、连接变化、属性变化）
- OCL（Object Constraint Language）——对象约束语言
- OCL 合约

3.23 面向过程的需求建模（结构化分析） （第 70–75 页）

- 结构化分析思想
- 数据流模型（行为视角）
- 实体关系模型（结构视角）
- 结构化分析方法概述
- 指导性原则
- 结构化方法的局限性
- 结构化分析工具：DFD、DD、结构化语言、判定表、判定树

3.24 数据流图（DFD） （第 71–72 页）

- 数据流图（Data Flow Diagram）
- 基本成份：外部实体、处理、数据流、数据存储
- 环境图（0 级 DFD / 顶层图）
- 数据流图的绘制步骤
- 分层数据流图
- 分层原则
- 特征与用途
- 教学管理系统分层数据流图示例

3.25 数据字典 （第 73 页）

- 数据字典（Data Dictionary）的定义与用途

- 数据字典的种类
- 数据字典的定义方法
- 数据字典——数据流
- 数据字典——数据项
- 数据字典——数据存储

3.26 结构化语言与判定工具 （第 73–74 页）

- 结构化语言
- 判定表（Decision Table）
- 判定树（Decision Tree）

3.27 图书借阅管理系统——完整 DFD 示例 （第 74–75 页）

- 顶层数据流图
- 中层数据流图
- 各模块底图（处理数据流图）

3.28 OO vs 面向过程建模对比 （第 76 页）

- 基于模型的系统需求建模——两种方法的比较

Part 5 需求确认与验证 （第 76–79 页）

3.29 需求验证 （第 76–77 页）

- 解决软件需求问题（遗漏、无源头、不一致、不正确、不规范）
- 需求验证目标
- Validation（确认）vs Verification（验证）——重要区分
- 需求确认与验证的手段:
  - 需求评审（Review）
  - 原型生成（Prototyping）
  - 测试用例生成（Test Case Generation）
- 软件需求评审的步骤
- 评审软件需求

3.30 需求规格说明 （第 78–79 页）

- 软件需求分析的输出：需求模型 + 软件原型 + 需求文档
- 需求规格说明（SRS）
- 用户需求报告
- 需求分析规格说明书
- 撰写软件需求文档的注意事项
- 软件需求文档模板

3.31 需求管理活动 （第 79 页）

- 需求工程管理活动（变更控制、版本控制、需求状态跟踪、需求追踪）

第 4 章：软件概要设计 （第 80–107 页）

Part 1 软件设计概述 （第 80–85 页）

4.1 软件设计的基本概念 （第 80–82 页）

- 从需求到设计和编码
- 软件设计的基本概念
- 何为软件设计?（定义、内容）
- 软件设计的内容
- 软件设计特点：多样性和差异性
- 需求分析、软件设计、软件实现间的关系
- 软件设计的质量要求：正确性、充分性、优化性、简单性

4.2 软件设计基本原则 （第 83–85 页）

1. **抽象原则**（Abstraction）——抽象在软件设计中的应用
   - 软件设计抽象层次的变化
   - 体系结构层次 / 构件层次的设计抽象
2. **模块化**（Modularity）——模块分解与开发成本的关系
3. **高内聚度原则**（High Cohesion）
   - 内聚度分类（7 种）：偶然内聚、逻辑内聚、时间内聚、过程内聚、通讯内聚、顺序内聚、功能内聚
4. **低耦合度原则**（Low Coupling）
   - 耦合度分类（7 种）：内容耦合、公共耦合、外部耦合、控制耦合、标记耦合、数据耦合、无直接耦合
5. **信息隐藏原则**（Information Hiding）——示例
6. **关注点分离原则**（Separation of Concerns）
7. **软件重用原则**（Software Reuse）

Part 2 软件设计过程 （第 85–89 页）

4.3 软件设计的过程 （第 85–87 页）

- 软件设计过程概述
- 软件设计元素：子系统、软件组件、设计类/设计函数
- 软件设计的过程——体系结构设计
- 分层体系结构风格示例
- 软件设计的过程——详细设计

4.4 软件设计的输出与评审 （第 88–89 页）

- 软件设计的输出：设计模型 + 设计文档
- 软件设计文档的规范
- 软件设计中的缺陷类型
- 软件设计的评审

- 软件设计的 CASE 工具

Part 3 软件架构设计 （第 89–94 页）

4.5 软件体系结构设计概述 （第 89–90 页）

- 软件架构设计概述
- 软件体系结构设计示意图
- 软件体系结构设计的目标
- 软件体系结构设计的任务
- 与软件需求的关系
- 与详细设计的关系
- 软件体系结构设计的过程（4 步骤）

4.6 设计初步的软件体系结构 （第 91–92 页）

- 设计初步的软件体系结构
- 不同体系结构模式适合的应用类型
- 重用开源软件及已有软件资产
- 到哪里找开源软件?（GitHub / SourceForge / Gitee）
- 示例：搜寻和重用软件资产

4.7 精化软件体系结构 （第 93–94 页）

- 精化软件体系结构
- 确定子系统及其接口
- 确定构件及其接口
- 优化体系结构设计
- 撰写软件体系结构设计文档
- 评审软件体系结构设计

Part 4 架构设计模型 （第 95–101 页）

4.8 面向对象的架构模型 （第 95–98 页）

- 回顾：面向对象的需求模型
- 软件体系结构的概念
- 面向对象软件设计过程
- **包图（Package Diagram）**
  - 包图的基本概念
  - 包的作用
  - 包的依赖关系
  - 示例：包图表示体系结构
- **构件图（Component Diagram）**
  - 组件的概念
  - 组件及其接口

  - 组件和接口的表示（供给接口 / 需求接口 / 端口）
  - 示例：软件组件（.dll / .jar）
  - 软件体系结构的逻辑视图：组件图

4.9 面向过程的架构模型 （第 99–101 页）

- 面向过程概要设计的主要内容
- H 图（层次图 / Hierarchy Chart）
- IPO 图（输入-处理-输出图）
- 结构图（Structure Chart）
- 面向数据流的设计方法
- 变换分析（Transform Analysis）
- 事物分析（Transaction Analysis）

Part 5 软件架构设计模式 （第 101–107 页）

4.10 软件体系结构风格 （第 101–102 页）

- 软件体系结构风格（Architecture Style）
- 建筑风格类比（中式/西式）
- 软件设计模式（Design Pattern）
- 常用软件体系结构风格概述

4.11 分层体系结构模式 （第 102–105 页）

- 分层体系结构模式（Layered Architecture）
- 分层模式的约束 / 特点
- 示例：网络 OSI 模型 / 云计算架构
- 三层结构设计（Three-Tier Architecture）
  - 表示层（Presentation Layer）
  - 中间层 / 业务逻辑层（Business Logic Layer）
  - 数据层（Data Layer）
- 三层协调工作 / 三层结构优点
- **MVC 风格**（Model-View-Controller）
  - Model（模型）、View（视图）、Controller（控制器）
  - MVC 风格的约束
- **MVP 模式**（Model-View-Presenter）
  - MVP 与 MVC 的区别
- **MVVM 框架**（Model-View-ViewModel）
- 贫血模型（Anemic Model）与充血模型（Rich Model）

4.12 管道与过滤器风格 （第 106 页）

- 管道与过滤器风格（Pipe-and-Filter）
- 模式的特点 / 约束
- 示例：Unix Shell 管道

4.13 SOA 与消息总线 （第 106–107 页）

- 面向服务的体系结构（SOA）
- SOA 风格的特点
- 消息总线风格（Message Bus）
- ESB（Enterprise Service Bus）体系结构

第 5 章：软件详细设计 （第 108–134 页）

Part 1 详细设计概述 （第 108–111 页）

5.1 详细设计的任务与定位 （第 108–110 页）

- 面向对象/面向过程方法的软件设计过程（回顾）
- 详细设计的动机
- 详细设计的任务与定位
- 目标与原则
- 主要活动
- **概要设计 vs 详细设计**（重要对比）：
  - 概要设计：做什么（What）——体系结构层面
  - 详细设计：怎么做（How）——实现细节层面
- 软件详细设计的要求和原则

5.2 软件详细设计的活动 （第 110–111 页）

1. 用例设计
2. 子系统/软构件设计
3. 类/函数设计
4. 持久化设计（数据设计）

Part 2 面向对象的详细设计过程 （第 111–126 页）

5.3 详细设计的过程（4 步骤） （第 112–113 页）

- 步骤 1：用例设计
- 用例设计的任务
- 系统与设计类间对应关系
- 设计元素及其交互
- 示例：用例顺序图 / 设计方案

5.4 类设计 （第 114–120 页）

- 类设计的任务
- 类设计原则：准确化、细节化、一致性、遵循设计原则
- 注意事项
- 类设计过程：
  1. 确定类的可见范围（public / protected / private）

2. 精化类间的关系（继承 / 组合 / 聚合 / 关联 / 依赖）
3. 精化类属性的设计
4. 精化类方法的设计
5. 构造类对象的状态图 / 活动图
6. 评审和优化

- **活动图（Activity Diagram）**
  - 基本概念
  - 组成元素：动作和活动节点、开始和终止、控制流、判断节点、分叉节点、合并节点、结合节点
  - 对象流（Object Flow）
  - 泳道（Swimlane）
- **状态图（State Machine Diagram）**
  - 概念、主要图符
  - 转移（Transition）：触发器 → 守卫 → 影响
- 类设计输出的软件制品

5.5 持久化设计（数据设计） （第 121–125 页）

- 为什么要进行持久化设计
- 数据设计的任务
- 数据设计的原则
- 持久化设计的任务
- 数据设计过程：
  1. 确定永久数据（哪些数据需要持久化存储）
  2. 确定持久数据的存储和组织方式
  3. 确定持久数据条目
  4. 关系映射：
     - 1:n 关联关系的映射（外键）
     - n:m 关联关系的映射（中间表）
     - 继承关系的数据库表设计（3 种策略）
  5. 设计数据操作（CRUD）
  6. 评审和优化数据设计
- 示例：设计持久数据
- 示例：设计永久数据的操作

5.6 设计整合与验证 （第 125–126 页）

- 软件详细设计的输出
- 设计整合
- 设计验证

Part 3 面向过程的详细设计 （第 126–133 页）

5.7 过程设计工具 （第 126–130 页）

- 面向过程详细设计概述
- **程序流程图（Flow Chart）**
  - 常见的程序流程图结构：顺序、选择、循环
- **N-S 图（Nassi-Schneiderman Diagram / 盒图）**
  - 特点：结构化、无 GOTO
- **PDL（Program Design Language / 程序设计语言）**
  - 伪代码形式
- **PAD（Problem Analysis Diagram / 问题分析图）**
- **决策表（Decision Table）**
- **判定树（Decision Tree）**

5.8 Jackson 方法（面向数据结构的设计方法） （第 130–133 页）

- Jackson 方法的基本思想
- Jackson 方法的 3 种基本类型：顺序、选择、重复
- Jackson 图（树形结构）
- 改进的 Jackson 图
- Jackson 方法解决问题的例子（零件库房管理）
- Jackson 方法步骤（1-5 步）
- 伪代码表示程序

Part 4 设计文档与评审 （第 133–134 页）

5.9 撰写与评审设计文档 （第 133–134 页）

- 撰写设计文档（目录结构）
- 软件设计方案提纲
- 优化和评审用例设计方案
- 设计评审人员（用户代表、设计人员、程序员、需求人员、质量人员、测试人员、配置管理员）
- 评审设计文档的要点：规范性、可追踪性、一致性、正确性、高质量、可实施性
- 评审步骤
- 小结

第 6 章：软件实现 （第 135–152 页）

Part 1 软件实现概论 （第 135–140 页）

6.1 软件实现的概念与任务 （第 135–136 页）

- 何为软件实现（微观概念 / 与设计和测试的关系）

- 软件实现的任务
- 软件实现与软件设计之间的关系
- 软件实现的特点：生产性活动 + 创作性活动

6.2 编码原则与规范 （第 137–138 页）

- 编写代码的原则（1/3）：易读、易改
- 编写代码的原则（2/3）：异常处理、代码与模型一致
- 编写代码的原则（3/3）：降低复杂度、软件重用
- 编程规范（Style Guide）
- 遵循编码风格（1/4–4/4）：格式化、简洁性、文档、结构化
- **代码重构（Code Refactoring）**——改善内部结构而不改变外部行为

6.3 软件实现的过程与工具 （第 139–140 页）

- 软件实现的过程：编码 + 单元测试 + 调试
- 支持软件实现的 CASE 工具——IDE
- 目前主流 IDE 及其特点：VSCode、JetBrains 系列、Eclipse
- 目前主流 IDE 发展历史
- 未来 IDE 特点与趋势：服务化、云端化、定制化

Part 2 编程语言 （第 140–143 页）

6.4 程序设计语言的类别 （第 140–141 页）

- 机器语言（Machine Language）
- 汇编语言（Assembly Language）
- 结构化程序设计语言（C、Pascal 等）
- 面向对象程序设计语言（Java、C++ 等）
- 描述性程序设计语言（SQL、HTML 等）
- 程序设计语言的表达能力分类：命令式 / 函数式 / 逻辑式

6.5 主流编程语言 （第 141–142 页）

- C / C++ 语言特点
- Java 语言特点
- Python 语言特点
- Swift / Kotlin / ArkTS / TypeScript / Rust / 仓颉
- 不同编程语言 Top 10 排名

6.6 程序设计语言的选择 （第 143 页）

- 软件的应用领域
- 与遗留软件系统的交互
- 软件的特殊功能及需求
- 软件的目标平台
- 程序员的编程经验

Part 3 软件编码过程 （第 143–149 页）

6.7 编写代码的任务 （第 144–145 页）

1. 编写实现类的代码
2. 编写实现类方法的代码
3. 编写实现类间关联的代码
4. 编写实现聚合和组合关系的代码
5. 编写实现接口关系的代码
6. 编写实现继承关系的程序代码
7. 编写实现包的代码

6.8 界面与数据编码 （第 146–147 页）

- 编写用户界面代码
- 示例：LoginUI 的编码实现
- 编写数据设计代码
- 示例：T_User 表的创建
- 示例：连接和关闭数据库的代码
- 示例：操作数据库的程序代码

6.9 框架与案例 （第 148–149 页）

- 什么是框架?（Framework）
- 框架 vs 类库的区别
- Case Study：基于 Java 技术栈的系统实现
- 例 1：基于 Java EE 的单体式应用实现（分层模型）
- 设计模型到 Java EE 代码的映射过程
- 类图 / 顺序图映射

6.10 微服务架构补充 （第 151–152 页）

- 微服务架构模型
- 基于 SpringCloud 的微服务应用
- 微服务组件图 / 概念类图 / 顺序图
- Spring Cloud 架构
- 服务发现配置
- 网关实现
- Spring Cloud 应用在阿里云部署

Part 4 软件文档 （第 149–150 页）

- 软件实现文档
- 用户指南的编写要求
- 用户指南的排版建议
- 用户指南的编写原则

- 软件实现管理文档
- 小结

第 7 章 / 第 6 章续：软件测试 （第 153–175 页）

Part 1 软件缺陷 （第 153–156 页）

7.1 软件缺陷基本概念 （第 153–154 页）

- 故障（Fault）分类：按发生阶段、系统边界、现象、目标、目的、持久性
- 软件缺陷（Software Defect / Bug）
- 故障（Fault）→ 错误（Error）→ 失效（Failure）——缺陷传播链
- 软件缺陷的危害（经济损失、人身安全等）

7.2 缺陷描述与管理 （第 155–156 页）

- 软件缺陷不可避免的原因
- 软件缺陷的描述：标识符、类型、严重程度、症状、修复优先级、状态、发现者、源头、原因
- 软件缺陷的应对方法：发现缺陷、修复缺陷、容忍缺陷、预防缺陷
- 程序调试（Debugging）——定义与步骤
- 缺陷管理工具（PingCode / Jira / ClickUp 等）

Part 2 软件测试概论 （第 157–160 页）

7.3 测试的基本概念 （第 157–158 页）

- 软件测试概论
- 在程序代码中找出软件缺陷
- 程序构成及其缺陷的潜在位置
- 软件测试的目的（发现错误、验证需求、提高质量）
- 软件测试的功效（不能证明程序无错）
- 软件测试的输入与输出

7.4 测试的本质与测试用例 （第 159 页）

- 软件测试的本质
- **测试用例（Test Case）**——四元组：
  1. 输入数据
  2. 前置条件
  3. 操作步骤
  4. 预期输出
- 示例：测试用例的设计（” 用户登录”）
- 软件测试的步骤（4 步）

7.5 软件测试原则 （第 160 页）

1. 测试应有计划、有组织地进行
2. 测试应追溯需求
3. Pareto 原则（80% 的错误来自 20% 的模块）
4. 穷举测试是不可能的
5. 第三方测试更有效
6. 测试用例应长期保留（回归测试用）
7. 避免测试自己编写的程序
8. 不仅要测试正常情况，还要测试异常情况

Part 3 软件测试分类 （第 160–166 页）

7.6 测试分类概述 （第 161 页）

- 静态测试 vs 动态测试
- 白盒测试 vs 黑盒测试 vs 灰盒测试
  - 白盒测试：知道内部结构，测试内部逻辑
  - 黑盒测试：不知道内部结构，测试功能
  - 灰盒测试：部分知道内部结构

7.7 白盒测试（White-Box Testing） （第 161–164 页）

- 白盒测试用例设计的指导原则
- **语句覆盖（Statement Coverage）**
  - 每条语句至少执行一次
- **判定覆盖（Branch Coverage / 分支覆盖）**
  - 每个判定的每个分支至少执行一次
- **组合覆盖（Condition Coverage）**
  - 每个条件的所有可能组合至少执行一次
- **基本路径测试（Basis Path Testing）**
  - 步骤 1：根据程序逻辑画出流程图
  - 步骤 2：将流程图转换为流图（Flow Graph）
    - 流图中的判定点不应含复合条件
  - 步骤 3：确定基本路径集合
    - 圈复杂度 V(G) = E - N + 2（或 P + 1，或判定节点数 + 1）
  - 步骤 4：针对测试路径设计测试用例
  - 步骤 5：运行程序检验测试用例

7.8 黑盒测试（Black-Box Testing） （第 165–166 页）

- 黑盒测试的思想、依据、特点
- **等价分类法（Equivalence Partitioning）**
  - 有效等价类
  - 无效等价类

  - 实例：等价类划分法（三角形问题）
- **边界值分析法（Boundary Value Analysis）**
  - 原则：边界值、边界值 ±1
  - 示例

Part 4 软件测试过程 （第 167–172 页）

7.9 软件测试活动层次 （第 167 页）

- 软件测试活动（5 个层次）：
  1. 单元测试（Unit Testing）
  2. 集成测试（Integration Testing）
  3. 确认测试（Validation Testing）
  4. 系统测试（System Testing）
  5. 验收测试（Acceptance Testing）

7.10 单元测试 （第 167–168 页）

- 程序单元测试的依据和对象
- 谁负责开展程序单元测试?（通常由开发人员）
- 单元测试技术（驱动模块 / 桩模块）
- 单元测试工具（JUnit / C++Test / pytest 等）

7.11 集成测试 （第 168–169 页）

- 集成测试的对象、技术、内容
- **集成测试策略**：
  - 大爆炸集成（Big Bang）——一次性集成所有模块
  - 自顶向下集成（Top-Down）——需要桩模块
  - 自底向上集成（Bottom-Up）——需要驱动模块
  - 三明治集成（Sandwich）——结合自顶向下和自底向上

7.12 确认测试与系统测试 （第 169–171 页）

- 确认测试（Validation Testing）——验证是否满足需求
- 功能测试
- **非功能性测试**：
  - 性能测试（Performance Testing）
  - 压力测试（Stress Testing）
  - 容量测试（Volume Testing）
  - 安全性测试（Security Testing）
  - 恢复测试（Recovery Testing）
  - 用户界面测试（UI Testing）
  - 兼容性测试（Compatibility Testing）
  - 可用性测试（Usability Testing）

7.13 验收测试与回归测试 （第 171–172 页）

- 验收测试（Acceptance Testing）
- α 测试（Alpha Testing）——开发方内部测试
- β 测试（Beta Testing）——用户方测试
- **回归测试（Regression Testing）**——修改后重新测试
  - 目的：确保修改未引入新缺陷
  - 方法：重新执行原有测试用例
- 测试过程与测试文档

Part 5 软件测试管理 （第 172–175 页）

7.14 测试组织与计划 （第 172–173 页）

- 成立软件测试组织（独立于开发团队）
- 软件测试的输出
- 制定和实施软件测试计划：
  - 标识符、测试项、测试策略
  - 交付物、任务、环境、进度、风险

7.15 软件测试八项流程 （第 173–175 页）

1. 熟悉项目情况
2. 排序测试需求——测试需求排序示例
3. 定义测试策略
4. 设计测试用例
5. 估计测试工作量
6. 配置测试资源
7. 执行测试
8. 书写测试报告

第 8 章：软件部署、维护与演化 （第 175–186 页）

Part 1 软件部署 （第 175–180 页）

8.1 软件部署的概念 （第 176 页）

- 何为软件部署（收集、打包、安装、配置、发布）
- 软件运行环境
- 安装和配置运行环境
- 安装和配置软件系统
- 软件与其运行环境的关系

8.2 部署的挑战与原则 （第 177 页）

- 软件间的关系：纵向层次性 / 横向相关性
- 软件运行环境的变化

- 软件部署的主要问题和挑战
- 软件部署的原则：
  1. 最小化原则
  2. 相关性原则
  3. 适应性原则

8.3 部署方式与方法 （第 178–180 页）

- 单机部署方式
- 分布式部署
- 软件部署方法：
  - 基于操作系统的部署
  - 基于软件中间件的部署
  - 基于容器的部署（Docker）
- **云原生（Cloud Native）**
- 基于部署图（Deployment Diagram）规划部署
- 部署工具：
  - Fat jar（基于语言的部署）
  - Docker（基于容器的部署）
  - Jenkins（基于流水线的部署）

Part 2 软件维护 （第 180–184 页）

8.4 软件维护的概念与特点 （第 180–181 页）

- 软件也需要进行维护
- 何为软件维护
- 软件维护的特点：同步性、周期长、费用高、难度大
- 软件维护成本模型：$M = P + K \cdot e^{(c-d)}$

8.5 软件维护分类 （第 181–182 页）

1. **纠错性维护（Corrective Maintenance）**——修复发现的缺陷
2. **适应性维护（Adaptive Maintenance）**——适应外部环境变化
3. **完善性维护（Perfective Maintenance）**——提升已有功能和性能
4. **预防性维护（Preventive Maintenance）**——预防潜在问题

8.6 维护的问题与可维护性 （第 182–183 页）

- 软件维护需要解决的问题：
  - 人员的问题
  - 软件制品的问题
  - 维护副作用的问题
- 软件的可维护性（4 个特性）：
  1. 可理解性（Understandability）
  2. 可测试性（Testability）

3. 可修改性（Modifiability）
4. 可移植性（Portability）

8.7 软件维护过程 （第 183–184 页）

- 软件维护过程
- 软件维护工作流程（12 步）
- 维护管理文档
- 维护文档

Part 3 软件演化 （第 184–186 页）

8.8 软件逻辑老化 （第 184 页）

- 何为软件逻辑老化（Software Aging）
- 软件逻辑老化的现象
- 软件逻辑老化的原因：
  - 缺乏变更
  - 质量下降
  - 变更成本增加
  - 负面变更
  - 用户满意度降低

8.9 软件演化及解决方法 （第 185–186 页）

- 软件演化及其特点（4 个特性）
- 解决方法：
  1. **代码重组（Code Restructuring）**
  2. **设计重构（Design Refactoring）**
  3. **逆向工程（Reverse Engineering）**——从代码反推设计
  4. **再工程（Re-engineering）**——重新设计再实现
- 小结

——全册目录结束——

共 8 章　186 页　祝考试顺利！